

AI Smart Booking System

A Working Prototype: AI-Powered Booking Workflow Built in n8n

• n8n • PostgreSQL • SQL • OpenRouter AI • Google Calendar API • Gmail / SMTP

1. Project Overview

The AI Smart Booking System is a working prototype built in n8n that automates part of the appointment-booking process using a conversational chat trigger, an LLM (via OpenRouter) for data extraction, a PostgreSQL database, Gmail/SMTP for confirmation emails, and the Google Calendar API for event creation.

The project was built and tested locally as a single n8n workflow, using n8n's built-in chat interface to simulate customer messages. It demonstrates a full pipeline from unstructured text input to a stored database record, an emailed confirmation, and a calendar entry — with several known limitations documented transparently in Section 10.

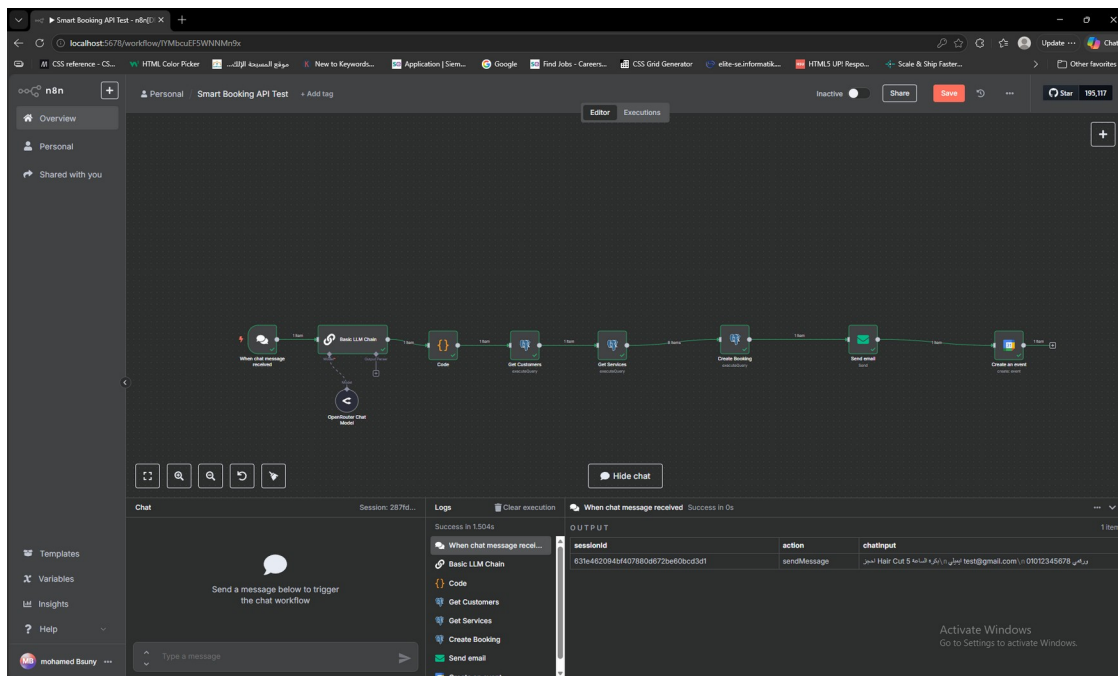


Figure 1 — Full n8n workflow: chat trigger → AI extraction → database → email → calendar.

2. Problem Statement

Manually handling service bookings — reading a message, checking availability, entering it into a system, and confirming it separately — is slow and repetitive for small service businesses. This project explores how far an AI + automation stack (n8n, an LLM, a relational database, and standard APIs) can go toward removing that manual work, using a self-contained, testable workflow.

3. Solution

The implemented pipeline works as follows:

- **1. Chat trigger** – A message is sent through n8n's built-in chat trigger (e.g. a booking request mixing Arabic and English text).
- **2. AI extraction** – An OpenRouter-hosted LLM (Basic LLM Chain node) receives the message and returns a JSON object with `service_name`, `booking_time`, `email`, and `phone`, extracted directly from the free-text message.
- **3. JSON parsing** – A Code node parses the JSON text returned by the model into usable fields for the rest of the workflow.
- **4. Database lookups** – Get Customers and Get Services nodes run SQL SELECT queries against PostgreSQL to read existing records.
- **5. Booking creation** – A Create Booking node runs an INSERT query, writing a new row into the bookings table (status defaults to "pending").
- **6. Confirmation email** – A Send Email node sends a fixed-text confirmation ("Your booking has been created successfully.") to the customer's email via Gmail/SMTP.
- **7. Calendar event creation** – A Create an Event node adds an event to Google Calendar via OAuth2.

Note: The calendar event's start/end time is currently set using `{{ $now }}` and `{{ $now.plus(1,'hour') }}` — i.e. the workflow's execution time — rather than the `booking_time` extracted from the customer's message. Tying the calendar event to the actual requested time is listed as a near-term improvement in Section 10.

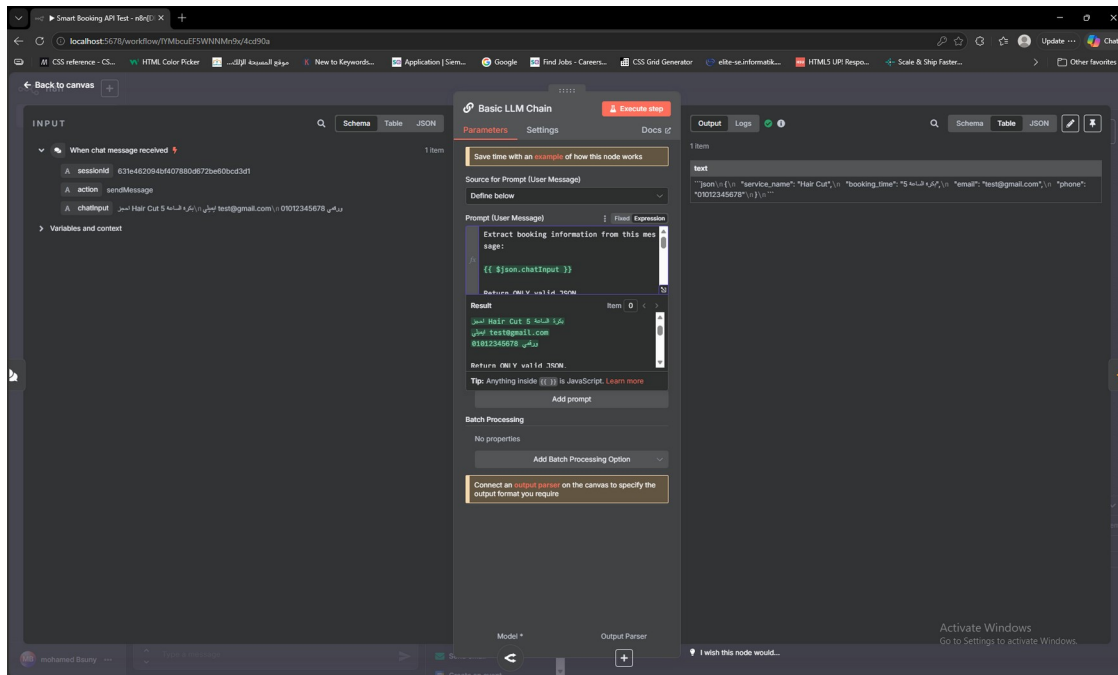


Figure 2 — AI extraction node: free-text message in, structured JSON out (`service_name`, `booking_time`, `email`, `phone`).

4. Architecture

The workflow is a linear sequence of n8n nodes, with no conditional branching implemented yet:

Chat Trigger → Basic LLM Chain (OpenRouter) → Code (parse JSON) → Get Customers (SELECT) → Get Services (SELECT) → Create Booking (INSERT) → Send Email (Gmail/SMTP) → Create an Event (Google Calendar, OAuth2)

Core Components

Layer	Technology	Role
Orchestration	n8n (local instance, localhost:5678)	Runs the linear node sequence; workflow currently set to Inactive (test mode)
NLU / Extraction	OpenRouter AI (Basic LLM Chain)	Extracts service_name, booking_time, email, phone as JSON from free text
Data Layer	PostgreSQL + SQL	customers, services, bookings tables; SELECT/INSERT used inside the workflow
Calendar	Google Calendar API (OAuth 2.0)	Creates a calendar event per booking (time not yet linked to booking_time)
Notifications	Gmail / SMTP	Sends a static confirmation message per booking

Current Design Characteristics

- **Linear flow** – The workflow runs top-to-bottom with no IF/Switch nodes — no conditional validation or conflict checking is currently implemented.
- **No error handling yet** – No Try/Catch or error-handling branches are present for the AI, database, email, or calendar calls.
- **Credentials** – Google OAuth 2.0 is used for Calendar access, so credentials aren't hard-coded in the workflow.

5. Database Design

Three related tables were created in PostgreSQL and are used by the workflow:

customers

Column	Type	Notes
id	bigint (PK)	Auto-incrementing identifier
name	text	Customer name
phone	text	Extracted from chat by the AI
email	text	Extracted from chat by the AI
created_at	timestamp	Row creation time

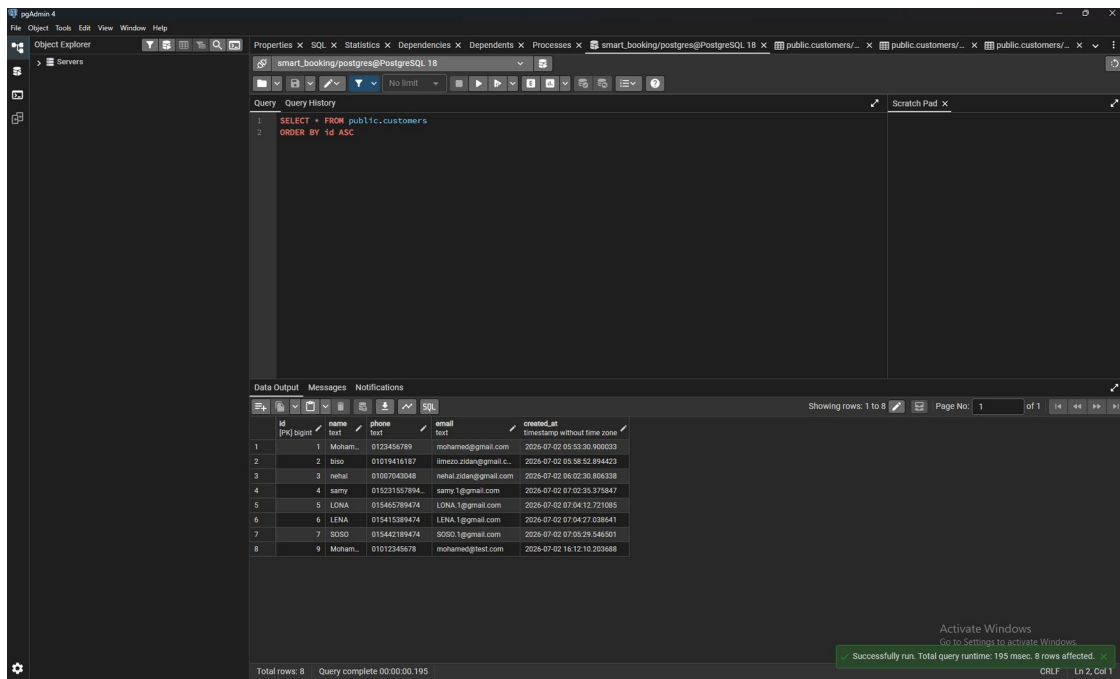


Figure 3 — customers table in PostgreSQL, populated from real workflow test runs.

services

Column	Type	Notes
id	bigint (PK)	Auto-incrementing identifier
service_name	text	Matched against AI-extracted service name
price	integer	Seed/test data
duration	integer	Seed/test data (minutes)

bookings

Column	Type	Notes
id	bigint (PK)	Auto-incrementing identifier
customer_id	bigint (FK → customers)	Links booking to a customer
service_id	bigint (FK → services)	Links booking to a service
booking_date	timestamptz	Currently set at workflow run time, not the AI-parsed time
status	text	Defaults to "pending"; no automated transition to "confirmed" yet
created_at	timestamp	Row creation time

Note: The services table currently mixes unrelated service types (e.g. haircut services alongside "Quran Memorization"), indicating this is seed/test data rather than a finalized catalog for one specific business.

SQL Operations — What's Actually Automated vs. Manual

Operation	Where it's used
SELECT	Inside the workflow — Get Customers and Get Services nodes
INSERT	Inside the workflow — Create Booking node
JOIN	Run manually in pgAdmin to review combined booking/customer/service data — not yet a workflow node
UPDATE	Performed manually in pgAdmin during testing (e.g. one row set to "confirmed") — not yet automated
DELETE	Not demonstrated in the current workflow or screenshots

The screenshot shows the pgAdmin 4 interface. The Query Editor contains the following SQL query:

```

1 SELECT *
2 FROM bookings b
3 JOIN customers c
4 ON b.customer_id = c.id
5 JOIN services s
6 ON b.service_id = s.id;

```

The Data Output pane displays the results of the query. The columns are: id, customer_id, service_id, booking_date, status, name, phone, email, created_at, service_name, price, and duration. The results show 17 rows of data.

Figure 4 — Manual JOIN query in pgAdmin combining bookings, customers, and services for reporting.

6. Features (Implemented)

- **Conversational booking intake** – Customers can send a free-text booking message (tested with mixed Arabic/English) through n8n's chat trigger.
- **AI-powered data extraction** – An OpenRouter LLM extracts service name, booking time (as text), email, and phone number into JSON.
- **JSON parsing** – A dedicated Code node parses the model's JSON output for use in later nodes.
- **Database lookups** – Customers and services are looked up in PostgreSQL via SELECT queries before a booking is created.
- **Automatic booking record creation** – A new row is inserted into the bookings table for every processed request.
- **Automated confirmation email** – A confirmation message is automatically emailed to the customer's address after booking creation.

- **Google Calendar event creation** – A corresponding event is automatically created on Google Calendar via OAuth2 for every booking.
- **Multi-service integration** – The workflow connects an LLM API, a relational database, an email service, and a calendar service in one automated pipeline.

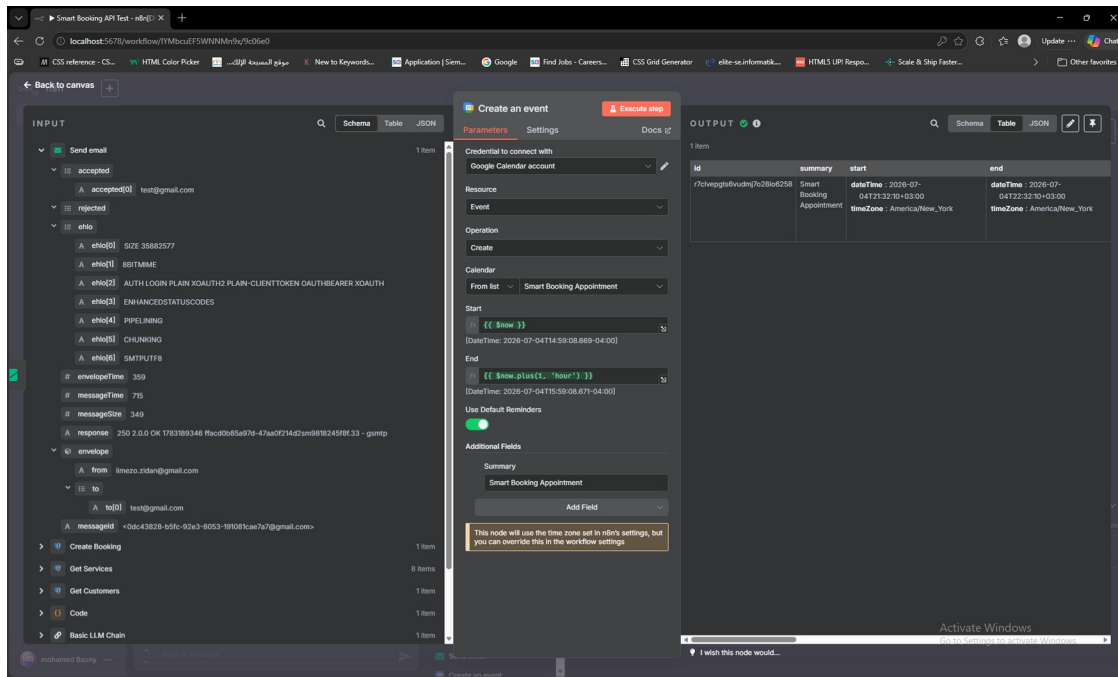


Figure 5 — Google Calendar event node configuration (OAuth2-authenticated).

7. Challenges

- **Reliable structured output from the LLM** – Getting the LLM to consistently return clean, parseable JSON (wrapped in markdown code fences) required a dedicated Code node to strip and parse the output.
- **Mixed-language input** – Test messages mixed Arabic and English in the same sentence; the AI extraction had to handle this without breaking field extraction.
- **OAuth credential setup** – Setting up OAuth 2.0 for Gmail and Google Calendar inside n8n so nodes could send email and create events without repeated manual re-authentication.
- **Multi-service orchestration** – Wiring together five different services (chat, LLM, database, email, calendar) into one working sequence, and iterating through repeated test executions (17 test bookings logged during development).

8. Results

- **End-to-end pipeline works for the happy path** – A single chat message reliably produces a database record, a confirmation email, and a calendar event, verified across 17 test executions.
- **Working relational data flow** – Customer and service data is read correctly from PostgreSQL and linked to new bookings via foreign keys.
- **Real third-party integrations** – Three services (LLM, email, calendar) are called successfully from within a single n8n workflow using real OAuth2 credentials.

Note: This is a local, test-mode prototype (workflow status: Inactive) rather than a deployed, production system. See Section 10 for what would be needed to take it further.

9. Skills Demonstrated

Category	Skills
Workflow Automation	n8n workflow design and node sequencing
AI Integration	Prompt writing for structured JSON extraction, OpenRouter API usage
Database Engineering	Relational schema design (3NF-style tables), SQL SELECT/INSERT, manual JOIN/UPDATE queries
API Integration	OAuth 2.0 setup for Gmail and Google Calendar APIs
Debugging & Testing	Iterative testing via n8n's execution logs; verifying data end-to-end in pgAdmin

10. Known Limitations & Roadmap

In the interest of accuracy, these are gaps that exist today, listed as planned future work rather than implemented features:

- **[Future] Real booking-time scheduling** – Calendar events should use the AI-extracted booking_time instead of the workflow's execution time.
- **[Future] Natural-language date/time parsing** – A free-text time expression (e.g. "tomorrow at 5") is not yet converted into a structured date/time value.
- **[Future] Conflict / double-booking detection** – No logic currently checks whether a requested time slot is already booked.
- **[Future] Status update / cancellation workflow** – Booking status is set once ("pending") with no automated path to "confirmed" or "cancelled."
- **[Future] Output validation before DB writes** – The AI's JSON output is not yet checked against a schema before being written to the database.
- **[Future] Detailed confirmation email content** – The confirmation email currently sends a fixed sentence; it should include the service, date, and time.
- **[Future] Error handling & retries** – No Try/Catch nodes exist yet for the LLM, database, email, or calendar calls.
- **[Future] Production deployment** – The workflow currently runs locally (localhost:5678) and is marked Inactive; it has not been deployed to a server.
- **[Future] Seed data cleanup** – The services table mixes unrelated services and should be cleaned up for a single, coherent business use case.

11. Portfolio-Ready Description

Built a working prototype of an AI-powered booking workflow in n8n that turns a free-text chat message into a database record, a confirmation email, and a Google Calendar event. The pipeline uses an OpenRouter-hosted

LLM to extract booking details as JSON, PostgreSQL to store customers/services/bookings with foreign-key relationships, and OAuth2-secured Gmail and Google Calendar integrations. Tested end-to-end across 17 sample bookings. Documented next steps include real time-slot scheduling, conflict detection, and production deployment.